

WHITE STORK

Migration Visualiser



Complete Application Guide

A full walkthrough of what the app does, how it is built, why every technology was chosen, and how the code works. Written for everyone — technical or not.

Ciconia ciconia · White Stork Portugal Adults 2022 · 3 Birds · 90 GPS Fixes · May 2022

CASTRO_O284	MINEIRO_O283	PERNETA_O285
0.09 km · 173 m · SE	2.72 km · 605 m · E	1.10 km · 184 m · N

What Is This Application?

The White Stork Migration Visualiser is a web application that tracks the real-life journeys of GPS-tagged wild storks — showing migration routes, stopovers, speed, altitude, and more through an interactive browser-based interface.

Think of it like Google Maps — but instead of tracking a car, it's tracking three individual storks across the Portuguese landscape in May 2022. Scientists fit the birds with backpack GPS tags (under 25g). Every 1–4 hours the tag records 30 values (position, speed, altitude, battery, temperature) and transmits them via mobile network to a research database. This app downloads that data, processes it, and presents it visually.

■ *In one line: Upload a GPS spreadsheet → the app draws the bird's route on a map, calculates stats, and serves it through a secure web interface that any browser can open.*

TECHNOLOGY

The Tech Stack — What Each Tool Is and Why It's Here

Six technologies work together. Each one explained in plain English — what it is, and the specific reason it was chosen for this bird-tracking app (not a generic 'it's popular' answer).

Python

Core Language

WHAT IT IS

A programming language — the instructions the computer runs. The entire application is written in Python. Think of it as the language this app speaks, like English is the language of this document.

WHY IN THIS APP

Python has the world's best scientific data libraries — pandas, geopandas, scipy, movingpandas. GPS coordinate analysis, distance calculation on a curved Earth, detecting stopovers — all built into Python libraries. Building this in another language would take months more work.

Flask

Web Framework

WHAT IT IS

A tool that turns a Python script into a website. Without Flask, Python cannot respond to a web browser. Flask listens on a port, receives requests ("give me the dashboard"), and sends back the right HTML or data.

WHY IN THIS APP

Flask is lightweight — it doesn't force a heavy project structure. For an academic project with 19 specific routes (login, map, admin, 6 API endpoints), Flask gives total control. Flask-Login handles authentication and Flask-SQLAlchemy handles the database connection with minimal setup.

MySQL

Database

WHAT IT IS

A database — a structured filing cabinet that stores information the app must remember between sessions: users, passwords, uploaded files. Without it, login is impossible and data resets every restart.

WHY IN THIS APP

The app needs to store user accounts with bcrypt-hashed passwords, track which files each user uploaded, and enforce two access roles (admin vs researcher). MySQL stores this reliably. SQLAlchemy lets Python talk to it via Python class definitions — no raw SQL required.

Pandas & GeoPandas

Data Processing

WHAT IT IS

Pandas is Excel inside Python — reads spreadsheets, filters rows, calculates means. GeoPandas is the same but understands coordinates and can measure real distances on the curved Earth surface, not just subtract numbers.

WHY IN THIS APP

The raw stork data is an Excel file with 99 rows × 30 columns. Pandas reads and cleans it in two lines. GeoPandas groups fixes by bird, sorts by timestamp, and uses Haversine geometry to measure actual flight distances. Without this, calculating '2.72 km' would require writing spherical geometry math from scratch.

GeoJSON

Map Format

WHAT IT IS

A text file format for storing geographic data — points, lines, and shapes with coordinates. Not a library, just a standard. A GeoJSON file is plain JSON with coordinates in a specific structure that map libraries understand natively.

WHY IN THIS APP

The frontend map needs to draw bird tracks. GeoJSON is the universal language all JavaScript map libraries speak. `preprocess.py` converts the Excel GPS data into four GeoJSON files (tracks, stopovers, stats, timeline). The browser reads these to draw lines and markers with no further calculation.

HTML / CSS / JavaScript

Frontend

WHAT IT IS

The three languages every browser runs. HTML is the structure (skeleton of each page), CSS is the styling (colours, layout), JavaScript is the behaviour (map interactions, chart updates, button clicks without page reload).

WHY IN THIS APP

The map page fetches GeoJSON from Flask's API via JavaScript — users click a bird name and the map updates instantly. `app.js` calls `/api/tracks` and `/api/stats`. `charts.js` draws speed and altitude timelines. Jinja2 (Flask's template engine) fills in dynamic values like the logged-in username before sending HTML.

How All the Pieces Connect

The application has three layers — browser, server, and storage. Here is how a request travels from the moment you open the app to the moment you see a bird's track on the map.

LAYER	COMPONENT	WHAT IT DOES
Browser (Frontend)	HTML + CSS + JavaScript app.js · charts.js	Shows the map, charts and dashboard to the user. Fetches data from Flask API without page reloads. Renders GeoJSON tracks on the interactive map.
Server (Flask)	backend/app.py 19 routes · Flask 3.1	Receives browser requests, checks login status, runs Python logic, queries the database, and returns HTML pages or JSON data.
Database (MySQL)	stork_db users · uploaded_files	Stores user accounts (hashed passwords, roles) and tracks which files each user uploaded. Persists data across all app restarts.
Files (Static)	static/geojson/ tracks · stopovers · stats · timeline	Pre-processed GeoJSON and JSON files. Generated once by preprocess.py from data.xlsx. Served directly by Flask API endpoints.

Project folder structure:

```
birdtracking/
├── backend/
│   ├── app.py           ← The server. All 19 routes live here.
│   ├── models.py        ← Database table definitions (User, UploadedFile).
│   ├── preprocess.py    ← Converts data.xlsx → GeoJSON + stats files.
│   └── analysis.py       ← Calculates distances, stopovers, headings.
├── data/
│   └── data.xlsx         ← Raw GPS data: 99 rows × 30 columns.
├── static/
│   ├── geojson/          ← tracks.geojson, stopovers.geojson, stats.json, timeline.json
│   ├── css/style.css     ← All visual styling.
│   ├── js/               ← app.js (map) + charts.js (timelines).
│   └── templates/        ← login.html, dashboard.html, map.html, admin.html ...
└── stork_db.sql          ← Database snapshot – import this to get started instantly.
```

DATA PIPELINE

From GPS Tag on a Bird's Back to a Dot on the Map

Raw GPS data goes through five clear transformation steps before you see anything on screen. Each step is handled by a different part of the codebase.

1

GPS Tag Records a Fix

Every 1–4 hours, the backpack-mounted tag locks onto satellites and records 30 values: latitude, longitude, altitude, ground speed, heading (compass direction), battery %, satellite count, accelerometer readings (detects wing flaps vs resting), external temperature, and more. The bird feels nothing — the tag weighs under 25g.

2

Data Exported as Excel File

The tag transmits each fix via GSM (mobile network) to the Movebank animal tracking database. Researchers export the data as data.xlsx — a spreadsheet with 99 rows (30 fixes per bird × 3 birds = 90 data rows + headers). This file is placed in the app's data/ folder.

3

preprocess.py Reads, Cleans and Transforms

When the app starts for the first time, preprocess.py runs automatically. It reads the Excel file with pandas, converts coordinates to a GeoPandas dataframe, groups fixes by bird name, sorts each group by timestamp, connects points into lines (LineString), detects stopovers (clusters where the bird barely moved), and calculates stats. Writes four files to static/geojson/. This heavy step only runs once — subsequent starts reuse the output.

4

Flask API Serves the Files on Demand

When a user opens the map page, the browser's JavaScript calls Flask API endpoints: /api/tracks, /api/stopovers, /api/stats, /api/timeline. Each endpoint simply opens the corresponding GeoJSON file and returns it as a JSON response. This separation means the browser downloads only the data it needs, when it needs it — not everything at once.

5

JavaScript Draws It on the Interactive Map

The browser receives the GeoJSON and uses a JavaScript map library to render bird tracks as coloured lines, stopovers as clickable circle markers, and individual GPS fixes as smaller dots. charts.js draws speed and altitude over time as line charts. Clicking a bird name filters the map without reloading the page.

The Key Code, Explained Line by Line

Four real code snippets from the application — each with a full explanation of what every line does and why it was written that way. No assumed knowledge.

1 — How a Route Works

backend/app.py · line 141

A 'route' links a URL to a Python function. When someone visits /dashboard, Flask runs this specific function.

```
@app.route('/dashboard')
@login_required
def dashboard():
    files = UploadedFile.query.filter_by(
        uploaded_by=current_user.id
    ).all()
    return render_template(
        'dashboard.html',
        files=files
    )
```

1 **@app.route('/dashboard')**

Registers this URL with Flask. When a browser visits /dashboard, Flask knows to run the dashboard() function. It's like giving an address to a building.

2 **@login_required**

Before running anything, Flask-Login checks for a valid session cookie. If the user is not logged in, they are automatically redirected to /login. No manual checking needed.

3 **def dashboard()**

The Python function that runs when the route matches. The function name is internal — only the @app.route string determines the URL.

4 **UploadedFile.query.filter_by(...).all()**

SQLAlchemy translates this into SQL: SELECT * FROM uploaded_files WHERE uploaded_by = [current user id]. Returns a Python list of file objects from MySQL.

5 **render_template('dashboard.html', files=files)**

Opens templates/dashboard.html, fills Jinja2 placeholders like {{ files }} with the real Python data, and sends the completed HTML page to the browser.

A 'model' is a Python class that describes a database table. SQLAlchemy reads it and creates the table in MySQL automatically — no SQL required.

```
class User(db.Model, UserMixin):
    id = db.Column(db.Integer,
                    primary_key=True)
    username = db.Column(db.String(80),
                          unique=True,
                          nullable=False)
    password = db.Column(db.String(255))
    role = db.Column(db.String(20),
                      default='researcher')

    def set_password(self, raw):
        self.password = bcrypt \
            .generate_password_hash(raw) \
            .decode('utf-8')

    def check_password(self, raw):
        return bcrypt.check_password_hash(
            self.password, raw
        )
```

1 **class User(db.Model)**

This Python class IS the database table. SQLAlchemy reads it on startup and creates a 'users' table in MySQL automatically — columns, types, constraints and all.

2 **primary_key=True**

Makes 'id' the unique row identifier. MySQL auto-increments it (1, 2, 3...) for every new user created. Required by every table.

3 **unique=True, nullable=False**

unique=True means no two users can share the same username — MySQL enforces this at database level. nullable=False means username cannot be blank. These constraints live in MySQL, not just Python.

4 **default='researcher'**

Every new user starts as a researcher. Only an existing admin can promote someone to admin through the admin panel. Prevents privilege escalation at registration.

5 **bcrypt.generate_password_hash(raw)**

Passwords are NEVER stored as plain text. bcrypt scrambles 'admin123' into a 60-character hash. Even if someone steals the database file, they cannot read back the original password.

6 **bcrypt.check_password_hash(self.password, raw)**

At login, bcrypt hashes what the user typed and compares it to the stored hash. No decryption happens — it simply verifies whether they match.

This is the core transformation: raw spreadsheet rows become the GeoJSON line that the map draws as a bird's migration route.

```
df = pd.read_excel(xlsx_path)

gdf = gpd.GeoDataFrame(df,
    geometry=gpd.points_from_xy(
        df['location-long'],
        df['location-lat']
    ),
    crs="EPSG:4326"
)

for bird_id, group in gdf.groupby(
    'individual-local-identifier'
):
    track = group.sort_values('timestamp')
    line = LineString(
        track.geometry.tolist()
    )
    tracks.append({
        "type": "Feature",
        "properties": {"bird_id": bird_id},
        "geometry": mapping(line)
    })
```

1 **pd.read_excel(xlsx_path)**

Opens data.xlsx and loads all 99 rows into memory as a pandas DataFrame — an in-memory spreadsheet. One line replaces hundreds of lines of file-parsing code.

2 **gpd.GeoDataFrame + points_from_xy()**

Converts the plain number columns location-long and location-lat into actual geometric Point objects that understand Earth's coordinate system.

3 **crs="EPSG:4326"**

Declares the coordinate system. EPSG:4326 is WGS84 — the same system GPS satellites use globally. This is required before GeoPandas can measure real-world distances.

4 **gdf.groupby('individual-local-identifier')**

Splits all 90 data rows into 3 groups — one per bird name. The for loop runs once per bird, processing each independently.

5 **group.sort_values('timestamp')**

Critical step. GPS fixes can arrive in any order. Sorting by time ensures the resulting line travels in the correct direction — not zigzagging randomly.

6 **LineString(...) → GeoJSON Feature**

Connects all sorted points into a single line geometry, then wraps it in the GeoJSON Feature format. This is what becomes one coloured track on the interactive map.

The map page fetches data dynamically — JavaScript calls these API routes and gets back JSON, which it then renders on the map without reloading.

```
def _read_geojson(filename):
    path = os.path.join(GEOJSON_DIR, filename)
    with open(path) as f:
        return json.load(f)

@app.route('/api/tracks')
@login_required
def api_tracks():
    return jsonify(
        _read_geojson('tracks.geojson')
    )

@app.route('/api/bird/<bird_name>')
@login_required
def api_bird(bird_name):
    stats = _read_geojson('stats.json')
    bird = stats['birds'].get(bird_name)
    if not bird:
        return jsonify({'error': 'not found'}), 404
    return jsonify(bird)
```

1 `_read_geojson(filename)`

A reusable helper that opens a file from `static/geojson/` and parses its JSON content into a Python dictionary. The underscore prefix is a Python convention meaning 'internal use only'.

2 `@app.route('/api/tracks')`

When JavaScript calls this URL, Flask returns the entire `tracks.geojson` as a JSON response. `jsonify()` sets the correct Content-Type header so the browser treats it as JSON, not HTML.

3 `/api/bird/`

A dynamic route parameter. If JavaScript calls `/api/bird/Castro_O284`, Flask extracts 'Castro_O284' and passes it as the `bird_name` variable. One route handles all three birds.

4 `stats['birds'].get(bird_name)`

Looks up the specific bird's stats. Using `.get()` instead of `[]` means if the bird name doesn't exist, it returns `None` instead of crashing the server.

5 `return jsonify({...}), 404`

Returns a proper 404 error response if the bird name is invalid. Good API design: errors should explain what went wrong, not crash or return empty data silently.

FEATURES

Every Feature, Explained

Migration Map Interactive map showing all three birds' GPS tracks as coloured lines, stopovers as circle markers, individual fixes as smaller dots. Click any element for details. Pan and zoom freely.	Dashboard Per-bird stat cards showing total distance, average speed, max speed, max altitude, stopover count, dominant heading, and tracking date range — all computed from GPS data.
Timeline Charts Line charts for ground speed and altitude over time for each bird. Shows when the bird was flying fast vs resting, and how altitude changed during the tracking period.	Secure Login Registration and login with bcrypt-hashed passwords. Session management via Flask-Login. Unauthenticated users are redirected to the login page automatically — no page is accessible without a valid session.
Admin Panel Admin-only page showing all registered users, their roles, and registration dates. Admins can change a user's role (admin ↔ researcher) or delete accounts. Protected by a custom <code>@admin_required</code> decorator.	Data Upload Upload a new Excel GPS file (up to 50 MB). The server runs <code>preprocess.py</code> on the new file, overwrites the GeoJSON output files, and the map updates with new data on next load.

SETUP

How to Run the App on Windows (Step by Step)

Five commands, run once. After that, only the last command each time to start the app.

1	Clone the code from GitHub <pre>git clone https://github.com/Nirmitt2002/birdtrekking.git cd birdtrekking</pre>
2	Install all Python packages <pre>pip install -r requirements.txt pip install pymysql cryptography</pre>
3	Add MySQL to PATH (PowerShell — do this each session) <pre>\$env:PATH += ";C:\Program Files\MySQL\MySQL Server 8.0\bin"</pre>
4	Create the database and import data (one time only) <pre>mysql -u root -p -e "CREATE DATABASE stork_db; CREATE USER 'stork_user'@'localhost' IDENTIFIED BY 's'" Get-Content stork_db.sql mysql -u root -p stork_db</pre>

5

Start the app — run this every time

```
python backend/app.py
```

Then open **http://localhost:5000** in any browser.

Login Credentials

Role	Username	Password	Access Level
Admin	admin	admin123	Full — all pages, user management, data upload, reprocess
Researcher	fenil	fenil123	Map, dashboard, charts — no admin panel or upload